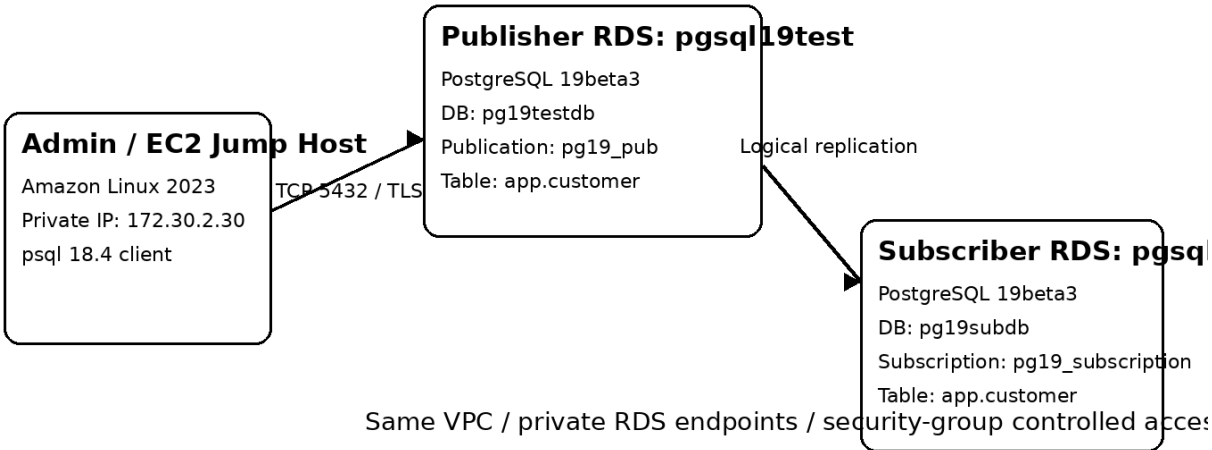


STANDARD OPERATING PROCEDURE

AWS RDS PostgreSQL 19 Preview
End-to-End Publisher / Subscriber Logical Replication



Document Type	Production-style technical SOP (implemented in RDS Preview lab)
Publisher	pgsql19test / pg19testdb
Subscriber	pgsql19sub / pg19subdb
AWS Region	us-east-2 (Ohio)
Database Engine	PostgreSQL 19 Preview (AWS RDS Preview)
Implementation Type	End-to-end tested logical replication validation
End-to-End Tested By	Praveen Madupu
Document Classification	Internal Technical SOP / Preview Validation
Status	Validated - Lab / Preview Environment

VALIDATION STATEMENT | End-to-End Tested by: Praveen Madupu

Security handling: Passwords, private keys, and secrets are intentionally replaced with placeholders. Rotate any credentials that were displayed during the lab implementation.

Table of Contents

1. Purpose and scope
 2. Target architecture and implemented inventory
 3. Prerequisites and security model
 4. EC2 administration host setup
 5. Publisher RDS setup
 6. Publisher database preparation
 7. PostgreSQL 19 validation checks
 8. Enable RDS logical replication
 9. Configure publication and replication user
 10. Subscriber RDS setup
 11. Subscriber database and schema preparation
 12. Create subscription
 13. Validate initial copy and real-time DML
 14. Monitoring and operational checks
 15. Change, rollback, and cleanup
 16. Production hardening recommendations
- Appendix A - Command reference
- Appendix B - Validation checklist

1. Purpose and Scope

This SOP documents the successful end-to-end implementation of PostgreSQL logical replication between two AWS RDS PostgreSQL 19 Preview DB instances using an EC2 administration host. It is written as a real-world operating procedure: infrastructure prerequisites, security-group design, database preparation, publication/subscription setup, validation, monitoring, and cleanup are covered in execution order.

- Publisher: pgsql19test, database pg19testdb, private endpoint pgsql19test.cpcosi8esiaa.us-east-2.rds-preview.amazonaws.com.
- Subscriber: pgsql19sub, database pg19subdb, private endpoint pgsql19sub.cpcosi8esiaa.us-east-2.rds-preview.amazonaws.com.
- Administration host: Amazon Linux 2023 EC2 instance in the same VPC, using PostgreSQL 18.4 client utilities to connect to PostgreSQL 19 Preview RDS Preview.
- Logical replication scope: app.customer table, publication pg19_pub, subscription pg19_subscription.
- Transport: private VPC routing, TCP 5432, TLS 1.3.

2. Target Architecture and Implemented Inventory

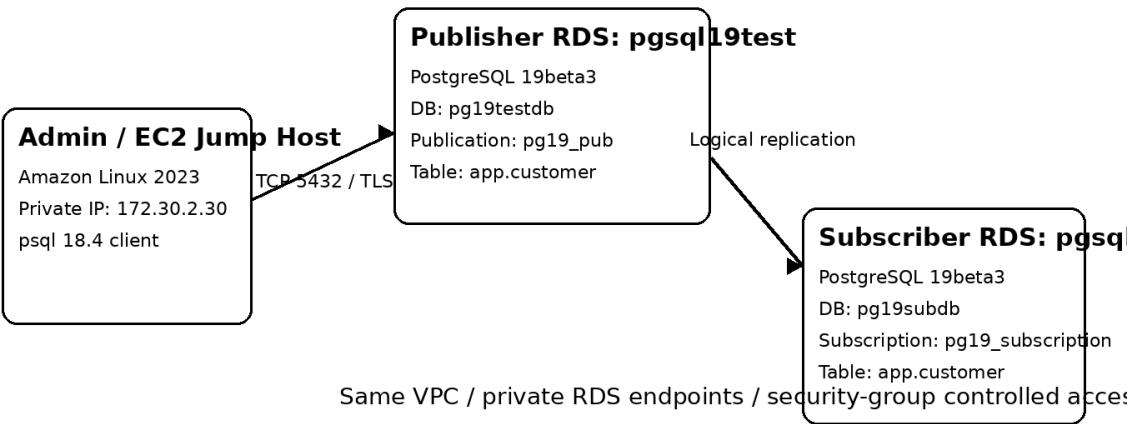


Figure 1 - Implemented logical replication architecture

Component	Name / Endpoint	Role	Key Details
EC2	ip-172-30-2-30	Admin / jump host	Amazon Linux 2023; psql 18.4
RDS Publisher	pgsql19test	Logical publisher	PostgreSQL 19 Preview; pg19testdb; 172.30.1.222
RDS Subscriber	pgsql19sub	Logical subscriber	PostgreSQL 19 Preview; pg19subdb; 172.30.2.158
Publication	pg19_pub	Change source	app.customer; INSERT/UPDATE/DELETE/TRUNCATE
Subscription	pg19_subscription	Change consumer	Enabled; pgoutput slot managed by subscription
Replication user	pg19repl	Dedicated publisher login	Member of rds_replication; SELECT on published table

3. Prerequisites and Security Model

- All resources are deployed in us-east-2 and use the same VPC so that database traffic remains private.
- RDS instances are not publicly accessible. Administration is performed through EC2.
- EC2 SSH is restricted to the administrator source IP. PostgreSQL 5432 is controlled with security-group references rather than 0.0.0.0/0.
- Publisher inbound TCP 5432 must allow the EC2 administration host and the subscriber-side network/security-group path.
- Passwords are never stored in scripts or documentation. Use AWS Secrets Manager or another approved secret store in production.

RDS behavior: The RDS master login named postgres is not a native PostgreSQL superuser. AWS-managed roles such as rds_replication are used for privileged replication operations.

4. EC2 Administration Host Setup

Step 4.1 - Launch an Amazon Linux 2023 EC2 instance

Create an EC2 instance in the same VPC as the RDS databases. A small instance class is sufficient for administration and validation workloads.

The screenshot shows the AWS Management Console 'Launch an instance' page. The browser address bar indicates the URL: `us-east-2.console.aws.amazon.com/ec2/home?region=us-east-2#LaunchInstances`. The page has a blue header with a message: 'It seems like you may be new to launching instances in EC2. Take a walkthrough to learn about EC2, how to launch instances and about best practices.' with buttons for 'Take a walkthrough' and 'Do not show me this message again.'.

The main content area is titled 'Launch an instance' with a sub-header 'Amazon Linux 2023 kernel-6.18 AMI'. Below this, there are several sections:

- Name and tags:** A text input field with 'e.g. My Web Server' and a button 'Add additional tags'.
- Application and OS Images (Amazon Machine Image):** A search bar with the text 'Search our full catalog including 1000s of application and OS images'. Below the search bar, there are tabs for 'Recents' and 'Quick Start'. Under 'Quick Start', there are icons for various operating systems: Amazon Linux, macOS, Ubuntu, Windows, Red Hat, SUSE Linux, and Debian. Below these icons, the 'Amazon Linux Image (AMI)' is selected, showing details for 'Amazon Linux 2023 kernel-6.18 AMI' (ami-06475c8f54266e38e) with a description of the OS and its support timeline.
- Summary:** A sidebar on the right containing:
 - Number of instances:** 1
 - Software Image (AMI):** Amazon Linux 2023 AMI 2023.12...read more
 - Virtual server type (instance type):** t3.micro
 - Firewall (security group):** New security group
 - Storage (volumes):** 1 volume(s) - 8 GiB

At the bottom right of the summary section, there are buttons for 'Cancel', 'Launch instance', and 'Preview code'.

Figure 2 - EC2 launch configuration using Amazon Linux 2023

Step 4.2 - Confirm VPC and instance network placement

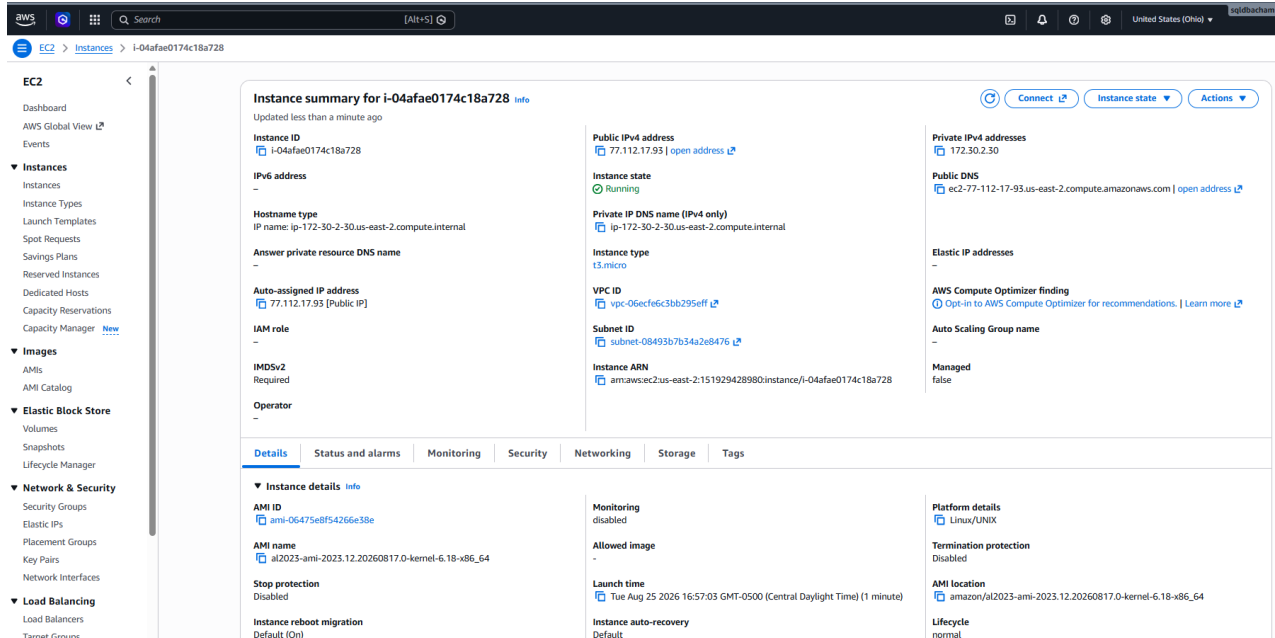


Figure 3 - EC2 instance details showing private IP, public IP, VPC, and subnet

Step 4.3 - Configure EC2 SSH security

Allow TCP 22 only from the administrator workstation/source IP. Do not expose SSH to all IPv4 addresses for an operational environment.

Step 4.4 - Connect from Windows PowerShell

```
ssh -i "T:\AWSRDS25Aug2026.pem" ec2-user@<EC2_PUBLIC_IP>
```

Windows private-key permissions: Windows OpenSSH requires the PEM file to be readable only by the intended account/administrators. Restrict inherited groups before connecting.

```
icacls "T:\AWSRDS25Aug2026.pem" /inheritance:r
icacls "T:\AWSRDS25Aug2026.pem" /remove "Authenticated Users"
icacls "T:\AWSRDS25Aug2026.pem" /remove "BUILTIN\Users"
icacls "T:\AWSRDS25Aug2026.pem" /grant:r "$($env:USERNAME):(R)"
```

Step 4.5 - Install PostgreSQL client tools on EC2

```
sudo dnf install postgresql18 -y
psql --version
```

Validated client version: psql 18.4. The RDS server version is PostgreSQL 19 Preview, so psql displays a major-version mismatch warning. Basic administration and logical-replication testing completed successfully.

5. Publisher RDS Setup - pgsql19test

Step 5.1 - Create the publisher RDS PostgreSQL 19 Preview instance

- DB identifier: pgsql19test
- Engine: PostgreSQL 19 Preview

- Port: 5432
- Publicly accessible: No
- VPC: same VPC as EC2 and subscriber
- Master username: postgres
- Use private subnets / DB subnet group appropriate to the VPC.

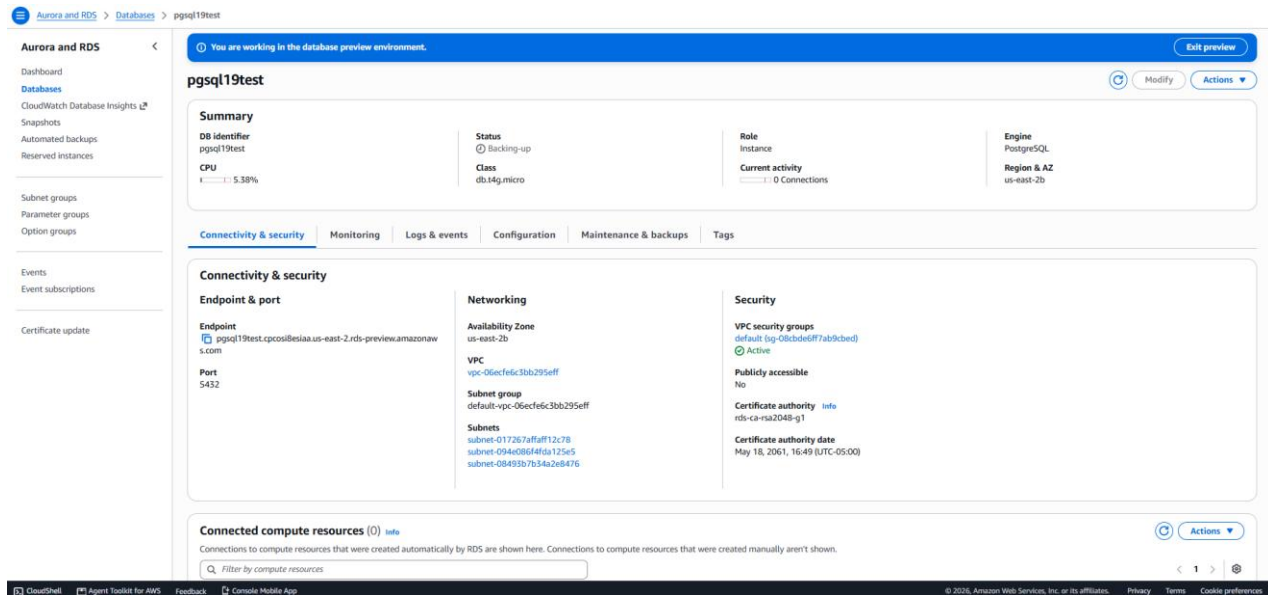


Figure 4 - Publisher RDS connectivity and security configuration

Step 5.2 - Configure RDS security group

Permit PostgreSQL TCP 5432 from the EC2 security group. For publisher-to-subscriber logical replication, also allow the subscriber security group (or shared RDS security group) as a source on TCP 5432.

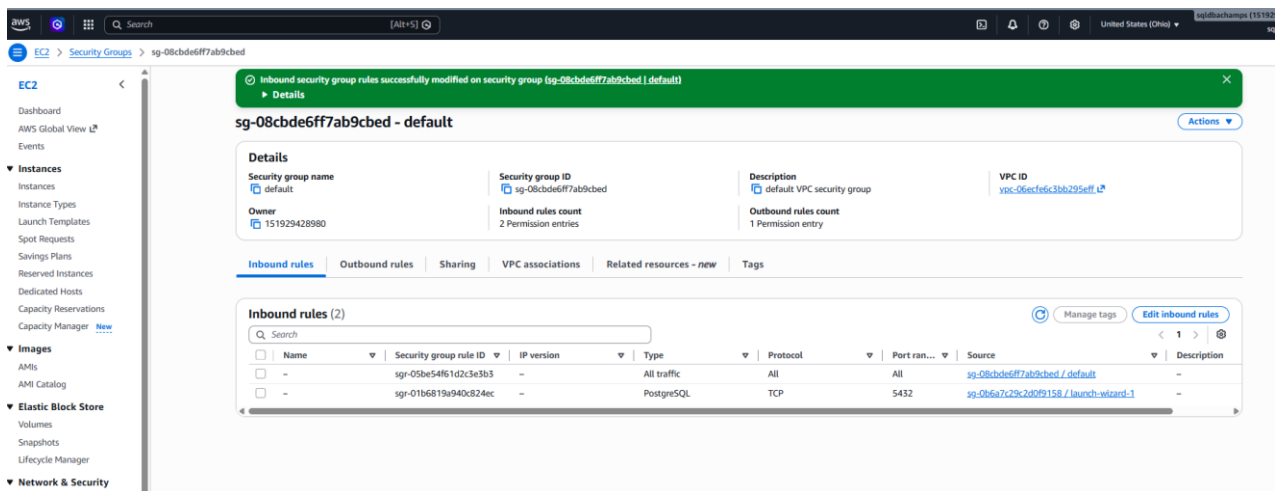


Figure 5 - RDS security-group inbound PostgreSQL rule using EC2 security-group source

Step 5.3 - Validate DNS and connectivity from EC2

```
getent hosts pgsql19test.cpcosi8esiaa.us-east-2.rds-preview.amazonaws.com
psql -h pgsql19test.cpcosi8esiaa.us-east-2.rds-preview.amazonaws.com -p 5432 -U postgres -d postgres
```

```
SELECT version();
SELECT current_database(), current_user, inet_server_addr(), inet_server_port();
\conninfo
```

Expected validation: The lab returned private address 172.30.1.222, port 5432, PostgreSQL 19 Preview, and TLS 1.3.

6. Publisher Database Preparation

Step 6.1 - Create application database and user

```
CREATE DATABASE pg19testdb;
CREATE USER pg19app WITH PASSWORD '<APP_PASSWORD>';
GRANT CONNECT ON DATABASE pg19testdb TO pg19app;
```

Step 6.2 - Reconnect to pg19testdb as postgres

```
psql -h pgsql19test.cpcosi8esiaa.us-east-2.rds-preview.amazonaws.com -p 5432 -U postgres -d pg19testdb
```

Step 6.3 - Create schema and table

```
CREATE SCHEMA app;
GRANT USAGE, CREATE ON SCHEMA app TO pg19app;

CREATE TABLE app.customer (
    customer_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    customer_name varchar(100) NOT NULL,
    email varchar(200),
    created_at timestamptz DEFAULT current_timestamp
);

GRANT SELECT, INSERT, UPDATE, DELETE ON TABLE app.customer TO pg19app;
GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA app TO pg19app;
```

Step 6.4 - Seed and validate application data

```
INSERT INTO app.customer (customer_name, email) VALUES
('Customer 1','customer1@gmail.com'),
('Customer 2','customer2@gmail.com');

SELECT * FROM app.customer ORDER BY customer_id;
```

Step 6.5 - Validate application login

```
psql -h pgsql19test.cpcosi8esiaa.us-east-2.rds-preview.amazonaws.com -p 5432 -U pg19app -d pg19testdb

SELECT current_user, current_database();
SELECT * FROM app.customer ORDER BY customer_id;
```

7. PostgreSQL 19 Preview Validation Checks

7.1 Exact server build evidence

The architecture and procedural text use “PostgreSQL 19 Preview” for clarity. The exact build observed during testing is retained below as command evidence.

```
SELECT version();
```

PostgreSQL 19beta3 on aarch64-unknown-linux-gnu, compiled by aarch64-unknown-linux-gnu-gcc (GCC) 12.4.0, 64-bit

These checks are not required for logical replication, but they were used to validate that the preview instance behaved as expected before configuring replication.

- pg_stat_statements 1.13 installed and query statistics collected.
- VACUUM (VERBOSE, ANALYZE) verified table maintenance and removal of dead tuples.
- Core PostgreSQL 19 REPACK command validated, including REPACK (CONCURRENTLY).
- SQL/PGQ property graph creation and GRAPH_TABLE single-hop/fixed two-hop matching validated.
- lag(...) IGNORE NULLS window-function behavior validated.
- Variable-length element-pattern quantifier was not supported by this beta build; it is not part of the replication implementation.

```
CREATE EXTENSION pg_stat_statements;
SELECT extname, extversion FROM pg_extension ORDER BY extname;

VACUUM (VERBOSE, ANALYZE) app.customer;
```

8. Enable Logical Replication on the Publisher

Step 8.1 - Confirm baseline parameters

```
SHOW wal_level;
SHOW max_replication_slots;
SHOW max_wal_senders;
SHOW rds.logical_replication;
```

Initial state in the lab: wal_level=replica, rds.logical_replication=off, max_replication_slots=20, max_wal_senders=35.

Step 8.2 - Create a custom PostgreSQL 19 DB parameter group

The screenshot shows the AWS RDS console interface for creating a new parameter group. The browser address bar indicates the URL: `us-east-2.console.aws.amazon.com/rds-preview/home?region=us-east-2#create-parameter-group`. The page title is "Create parameter group". A blue banner at the top states "You are working in the database preview environment." with an "Exit preview" button. The main section is titled "Create parameter group" and contains a form with the following fields:

- Parameter group details**
 - Parameter group name:** `pg19-logical-replication` (with a note: "The name must have 1 to 255 characters and begin with a letter. The name can't end with a hyphen or contain two consecutive hyphens. Valid characters: A-Z, a-z, 0-9, and - (hyphen)").
 - Description:** `PostgreSQL 19 logical replication testing` (with a note: "This description appears in the Parameter groups dashboard. You can use it to quickly identify the purpose of the parameter group.")
 - Engine type:** `PostgreSQL` (dropdown menu).
 - Parameter group family:** `postgres19` (dropdown menu, with a note: "You can associate a DB parameter group with only one DB parameter group family. You can apply the parameter group only to a DB instance whose DB engine is compatible with the parameter group family.")
 - Type:** `DB Parameter Group` (dropdown menu, with a note: "Type for the DB parameter group")

At the bottom right of the form are "Cancel" and "Create" buttons.

Figure 6 - Create custom DB parameter group pg19-logical-replication

Step 8.3 - Set rds.logical_replication = 1

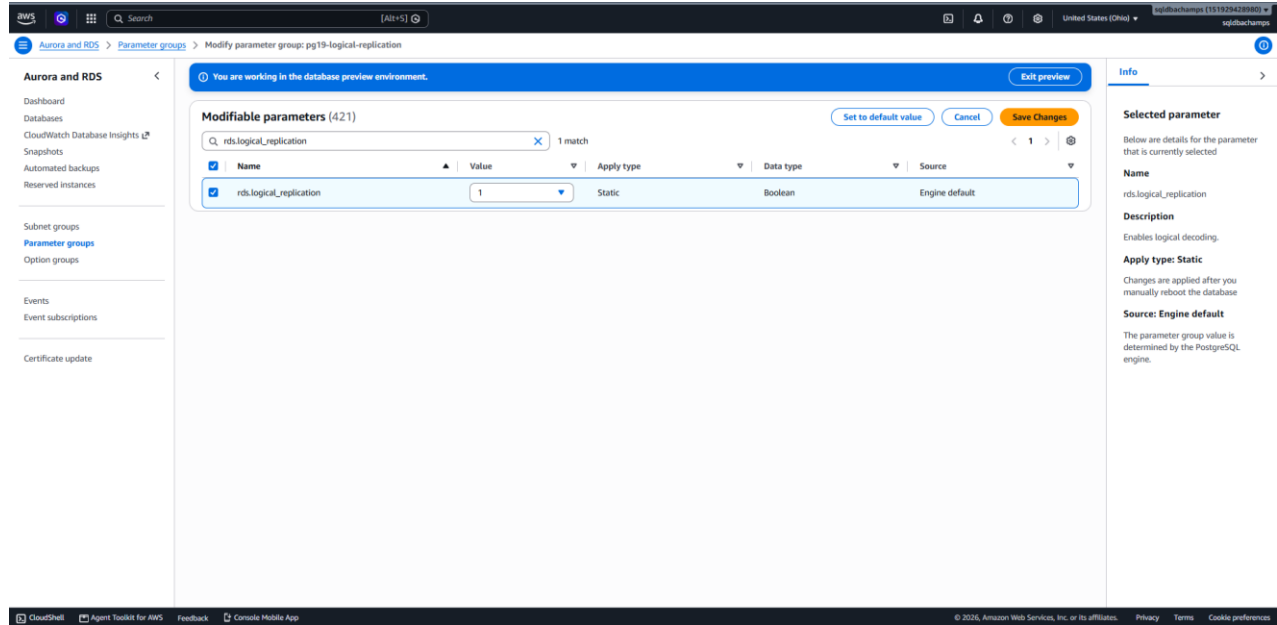


Figure 7 - Static RDS logical-replication parameter set to 1

Step 8.4 - Attach the custom parameter group to pgsq19test

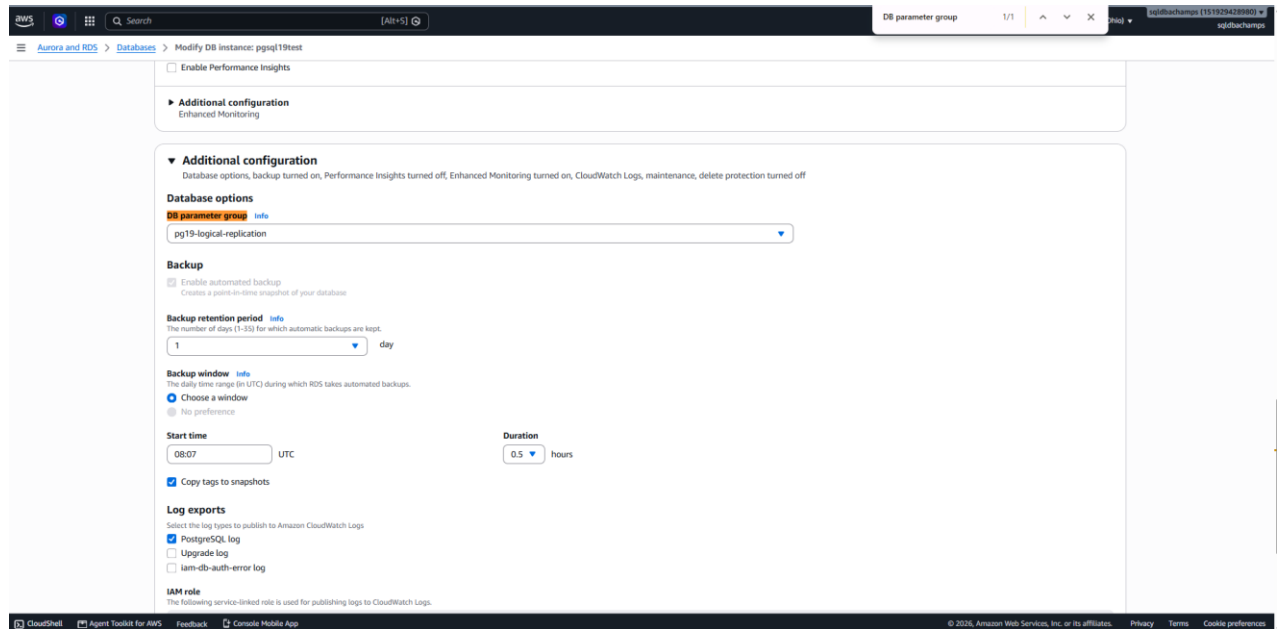


Figure 8 - Assign custom DB parameter group to publisher instance

Step 8.5 - Apply immediately and reboot when status is Pending reboot

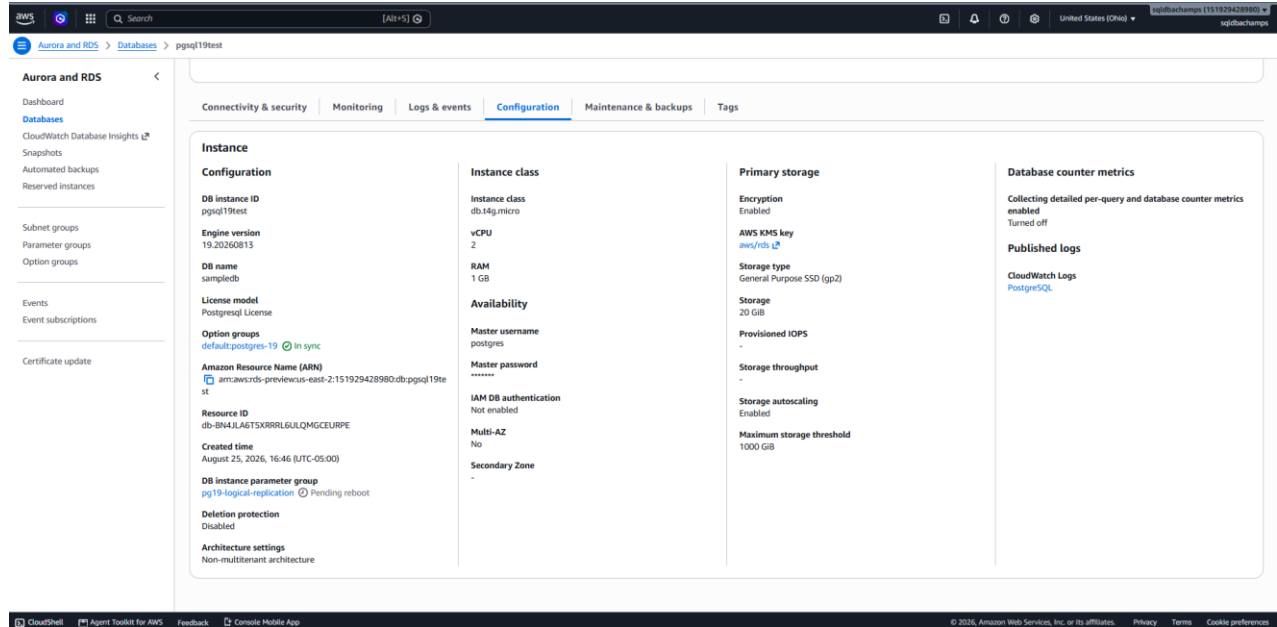


Figure 9 - Publisher parameter group in Pending reboot state

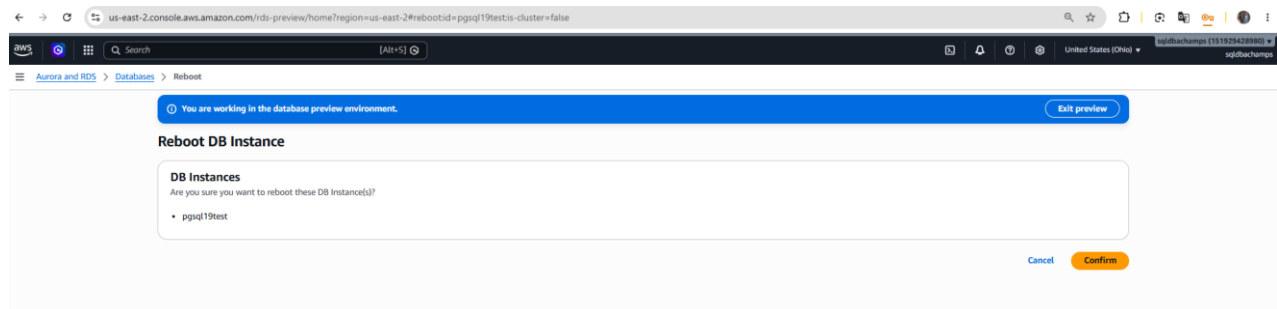


Figure 10 - Confirm publisher RDS reboot

Step 8.6 - Validate effective parameters after reboot

```
SHOW rds.logical_replication;
SHOW wal_level;
SHOW max_replication_slots;
SHOW max_wal_senders;
```

Expected values: rds.logical_replication=on and wal_level=logical. In the lab, max_replication_slots=20 and max_wal_senders=35 remained available.

9. Configure Publication and Replication User on Publisher

Step 9.1 - Create publication

```
CREATE PUBLICATION pg19_pub
FOR TABLE app.customer;

SELECT pubname, pubinsert, pubupdate, pubdelete, pubtruncate
FROM pg_publication;
```

```
SELECT pubname, schemaname, tablename
FROM pg_publication_tables;
```

Step 9.2 - Create dedicated replication login

```
CREATE USER pg19repl WITH PASSWORD '<REPLICATION_PASSWORD>';
GRANT rds_replication TO pg19repl;
GRANT CONNECT ON DATABASE pg19testdb TO pg19repl;
GRANT USAGE ON SCHEMA app TO pg19repl;
GRANT SELECT ON TABLE app.customer TO pg19repl;
```

Step 9.3 - Validate RDS replication role membership

```
SELECT pg_has_role('pg19repl', 'rds_replication', 'MEMBER') AS has_rds_replication;

SELECT current_user, current_database();
```

Step 9.4 - Validate login as replication user from EC2

```
psql -h pgsql19test.cpcosi8esiaa.us-east-2.rds-preview.amazonaws.com -p 5432 -U pg19repl -d pg19testdb

SELECT * FROM app.customer ORDER BY customer_id;
```

10. Subscriber RDS Setup - pgsql19sub

Step 10.1 - Create second PostgreSQL 19 Preview RDS instance

- DB identifier: pgsql19sub
- Engine/version: same PostgreSQL 19 Preview build as publisher
- Port: 5432
- VPC: same as publisher/EC2
- Publicly accessible: No
- Use a security group that allows EC2 administration on 5432 and allows outbound access to the publisher.

Parameter group: The subscriber does not need `rds.logical_replication` enabled merely to consume changes. Keep the default PG19 parameter group unless the subscriber will also act as a publisher.

Step 10.2 - Validate subscriber DNS and psql connectivity from EC2

```
getent hosts pgsql19sub.cpcosi8esiaa.us-east-2.rds-preview.amazonaws.com
psql -h pgsql19sub.cpcosi8esiaa.us-east-2.rds-preview.amazonaws.com -p 5432 -U postgres -d postgres
```

Expected validation: The implemented subscriber resolved to private address 172.30.2.158 and accepted TLS 1.3 connections.

11. Subscriber Database and Schema Preparation

Step 11.1 - Create subscriber database

```
CREATE DATABASE pg19subdb;
```

Step 11.2 - Reconnect to pg19subdb and create matching schema/table

```
psql -h pgsql19sub.cpcosi8esiaa.us-east-2.rds-preview.amazonaws.com -p 5432 -U postgres -d pg19subdb
CREATE SCHEMA app;
CREATE TABLE app.customer (
    customer_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    customer_name varchar(100) NOT NULL,
    email varchar(200),
    created_at timestamptz DEFAULT current_timestamp
);
SELECT * FROM app.customer ORDER BY customer_id;
```

Expected state: The subscriber table must exist before CREATE SUBSCRIPTION and should be empty so the initial copy is unambiguous. PostgreSQL logical replication does not deploy normal table DDL for you.

12. Create the Logical Subscription

Step 12.1 - Connect to subscriber pg19subdb as postgres

```
psql -h pgsql19sub.cpcosi8esiaa.us-east-2.rds-preview.amazonaws.com -p 5432 -U postgres -d pg19subdb
```

Step 12.2 - Create subscription

```
CREATE SUBSCRIPTION pg19_subscription
CONNECTION 'host=pgsql19test.cpcosi8esiaa.us-east-2.rds-preview.amazonaws.com port=5432
dbname=pg19testdb user=pg19repl password=<REPLICATION_PASSWORD> sslmode=require'
PUBLICATION pg19_pub;
```

Secret handling: Use the real password only at execution time. In production, do not leave credentials in shell history, tickets, or static documents.

Step 12.3 - Validate subscription catalog and worker

```
SELECT subname, subenabled, subslotname, subsynccommit
FROM pg_subscription;

SELECT * FROM pg_stat_subscription;
```

Successful implementation produced subscription pg19_subscription, enabled=true, with an active apply worker and a publisher-side slot named pg19_subscription.

Step 12.4 - Validate initial table copy

```
SELECT * FROM app.customer ORDER BY customer_id;
```

The subscriber received the existing publisher rows for customer IDs 2, 3, and 4.

13. Validate Real-Time INSERT / UPDATE / DELETE

Operational rule: Application writes are issued on the publisher only. Direct writes on the subscriber can create divergence and sequence conflicts in a one-way logical-replication topology.

Step 13.1 - INSERT on publisher

```
-- Run on pgsq19test / pg19testdb
INSERT INTO app.customer (customer_name, email)
VALUES ('Customer 5','customer5@gmail.com')
RETURNING *;
```

Step 13.2 - Verify INSERT on subscriber

```
-- Run on pgsq19sub / pg19subdb
SELECT * FROM app.customer WHERE customer_id = 5;
```

Step 13.3 - UPDATE on publisher and verify subscriber

```
-- Publisher
UPDATE app.customer
SET email='customer5_updated@gmail.com'
WHERE customer_id=5;

-- Subscriber
SELECT * FROM app.customer WHERE customer_id=5;
```

Step 13.4 - DELETE on publisher and verify subscriber

```
-- Publisher
DELETE FROM app.customer WHERE customer_id=5;

-- Subscriber
SELECT * FROM app.customer WHERE customer_id=5;
```

Observed result: INSERT, UPDATE, and DELETE changes made on the publisher were visible on the subscriber. The final DELETE validation returned zero rows for customer_id=5.

1. 6. Operations validates freshness using pg_stat_subscription and pg_replication_slots; CloudWatch/RDS metrics provide infrastructure health.
2. 5. Subscriber apply worker writes the replicated DML to pgsq19sub / pg19subdb.
3. 4. Publisher slot pg19_subscription tracks consumption and WAL retention.
4. 3. Subscription pg19_subscription connects with the dedicated pg19repl account over TLS on TCP 5432.
5. 2. Publication pg19_pub exposes changes for app.customer.
6. 1. Application writes only to pgsq19test / pg19testdb.

Scenario: An application transaction is committed on the publisher. PostgreSQL writes the change to WAL, the logical decoding/pgoutput path exposes the change through the publication and replication slot, and the subscriber apply worker commits the change to the matching target table.

13A. Real-Time Operational Scenario

14. Monitoring and Operational Checks

Step 14.1 - Subscriber worker health

```
SELECT subname, worker_type, pid, received_lsn, last_msg_send_time,
       last_msg_receipt_time, latest_end_lsn, latest_end_time
FROM pg_stat_subscription;
```

Step 14.2 - Publisher replication-slot health

```
SELECT slot_name, plugin, slot_type, database, active, restart_lsn, confirmed_flush_lsn
FROM pg_replication_slots;
```

Step 14.3 - Measure retained WAL

```
SELECT slot_name, active, restart_lsn, confirmed_flush_lsn,
       pg_current_wal_lsn() AS current_wal_lsn,
       pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn)) AS retained_wal
FROM pg_replication_slots;
```

Operational risk: Inactive logical slots retain WAL. Before the subscription was created, the manual test slot grew from about 64 MB to about 896 MB retained WAL. Remove unused slots promptly.

Step 14.4 - Recommended CloudWatch/RDS monitoring

- FreeStorageSpace and transaction-log/WAL growth indicators
- DatabaseConnections
- CPUUtilization
- FreeableMemory
- Read/Write IOPS and latency
- RDS PostgreSQL logs exported to CloudWatch
- Application-level replication freshness checks

15. Change, Rollback, and Cleanup

Step 15.1 - Disable subscription when troubleshooting

```
ALTER SUBSCRIPTION pg19_subscription DISABLE;
```

Step 15.2 - Re-enable subscription

```
ALTER SUBSCRIPTION pg19_subscription ENABLE;
```

Step 15.3 - Remove subscription

```
DROP SUBSCRIPTION pg19_subscription;
```

Dropping a normal subscription attempts to remove its associated publisher replication slot. Validate publisher `pg_replication_slots` afterward.

Step 15.4 - Remove publication if no longer needed

```
DROP PUBLICATION pg19_pub;
```

Step 15.5 - Remove dedicated replication account if no longer required

```
DROP ROLE pg19repl;
```

Step 15.6 - Revert logical-replication parameter if required

Set `rds.logical_replication` back to 0 in the custom parameter group, apply, and reboot. Confirm `wal_level` returns to the expected non-logical value before closing the change.

- Segregation of duties: production approval and execution should follow the organization's normal DBA/cloud operations control model.
- Recovery: maintain current backups/snapshots and a documented subscription rebuild procedure before production change execution.
- Monitoring: alert on subscriber worker failures, abnormal WAL retention, storage pressure, connectivity errors, and replication freshness thresholds.
- Evidence: retain parameter-group screenshots, RDS configuration screenshots, SQL validation output, subscription health output, and test results.
- Access control: use named administrator access, least-privilege database roles, restricted security-group references, and approved secret storage.
- Change control: record approved change/request ID, implementation window, owner, rollback decision point, and validation evidence.

The following controls should be attached to the implementation/change record for an enterprise deployment.

15A. IT Operational Controls and Evidence

16. Production Hardening Recommendations

- Use PostgreSQL GA versions for production. The PostgreSQL 19 Preview build and the RDS Preview environment used in this SOP are suitable for validation, not production service.
- Use a matching psql/pg_dump major version for production-grade administration and backup validation when available.
- Keep both RDS instances private and restrict 5432 to required security groups only.
- Store database and replication passwords in AWS Secrets Manager and rotate regularly.
- Use separate RDS security groups for publisher and subscriber in production so each traffic direction is explicit.
- Use least-privilege replication users and table grants; avoid using the RDS master login for subscription connections.
- Monitor replication slots, apply workers, storage, WAL retention, and subscriber freshness.
- Automate schema deployment independently of logical data replication.
- Define failover behavior: one-way logical replication does not automatically provide transparent application failover or bidirectional write safety.
- Validate sequence/identity behavior before promoting a subscriber to a writable target.
- Back up both sides and rehearse subscription rebuild procedures.

Appendix A - Command Reference

```
# Publisher connect
psql -h pgsql19test.cpcosi8esiaa.us-east-2.rds-preview.amazonaws.com -p 5432 -U postgres -d pg19testdb

# Subscriber connect
psql -h pgsql19sub.cpcosi8esiaa.us-east-2.rds-preview.amazonaws.com -p 5432 -U postgres -d pg19subdb

# Replication state
SHOW rds.logical_replication;
SHOW wal_level;
SELECT * FROM pg_publication;
SELECT * FROM pg_stat_subscription;
SELECT slot_name, active, restart_lsn, confirmed_flush_lsn FROM pg_replication_slots;
```

Appendix B - Implementation Validation Checklist

Check	Result	Operator Notes
EC2 host in correct VPC	PASS	
SSH access restricted and working	PASS	
PostgreSQL client installed	PASS	
Publisher private endpoint reachable	PASS	
Publisher application DB/schema/table created	PASS	
Application account CRUD validated	PASS	
Custom RDS parameter group attached	PASS	
rds.logical_replication=on	PASS	
wal_level=logical	PASS	
Publication pg19_pub created	PASS	
Dedicated pg19repl login validated	PASS	
Subscriber private endpoint reachable	PASS	
Subscriber pg19subdb and target table created	PASS	
Subscription pg19_subscription created	PASS	
Initial data copy completed	PASS	
Apply worker visible in pg_stat_subscription	PASS	
INSERT replicated	PASS	
UPDATE replicated	PASS	
DELETE replicated	PASS	
Unused manual replication slot removed	PASS	

Document closeout: This SOP represents the successful configuration path from EC2 administration host and RDS publisher creation through an operational one-way logical replication subscription and DML validation. Errors from the interactive lab are deliberately excluded from the procedural steps.

Document Sign-Off

End-to-End Tested By	Praveen Madupu
Technical Result	PASS - Publisher-to-subscriber logical replication validated
Environment	AWS RDS PostgreSQL 19 Preview / us-east-2
Production Note	Preview validation only; production adoption requires approved GA release and enterprise change controls